

Configure Allowed HTTP Methods Globally (Middleware + ActionFilter)

What is "Configure Allowed HTTP Methods Globally"?

Restricting which HTTP methods (GET, POST, PUT, DELETE, PATCH, etc.) your API accepts at a **global level**, before any request reaches your controllers.

Why it matters: Instead of checking HTTP methods in every controller action, you configure it ONCE and it applies to ALL requests.

Two Approaches in This Project

Approach	Where It Runs	Best For
Middleware	Before routing (very early in pipeline)	Simple global blocking, best performance
ActionFilter	After routing, before action execution	When we need access to controller/action context

Both approaches are included in this project for comparison.

Now we are going to set up a project 'ClothsApi5_ConfigureHttpMethods' to represent the above concepts,

Project Setup

Create the project:

Command: dotnet new webapi -n ClothsApi5_ConfigureHttpMethods -f net8.0

Navigate in the project:

Command: cd ClothsApi5_ConfigureHttpMethods

Install Packages

Before installing the packages, we should make sure that we are in the project folder 'ClothsApi5_ConfigureHttpMethods' in terminal. If not, then navigate to the exact project folder in terminal.

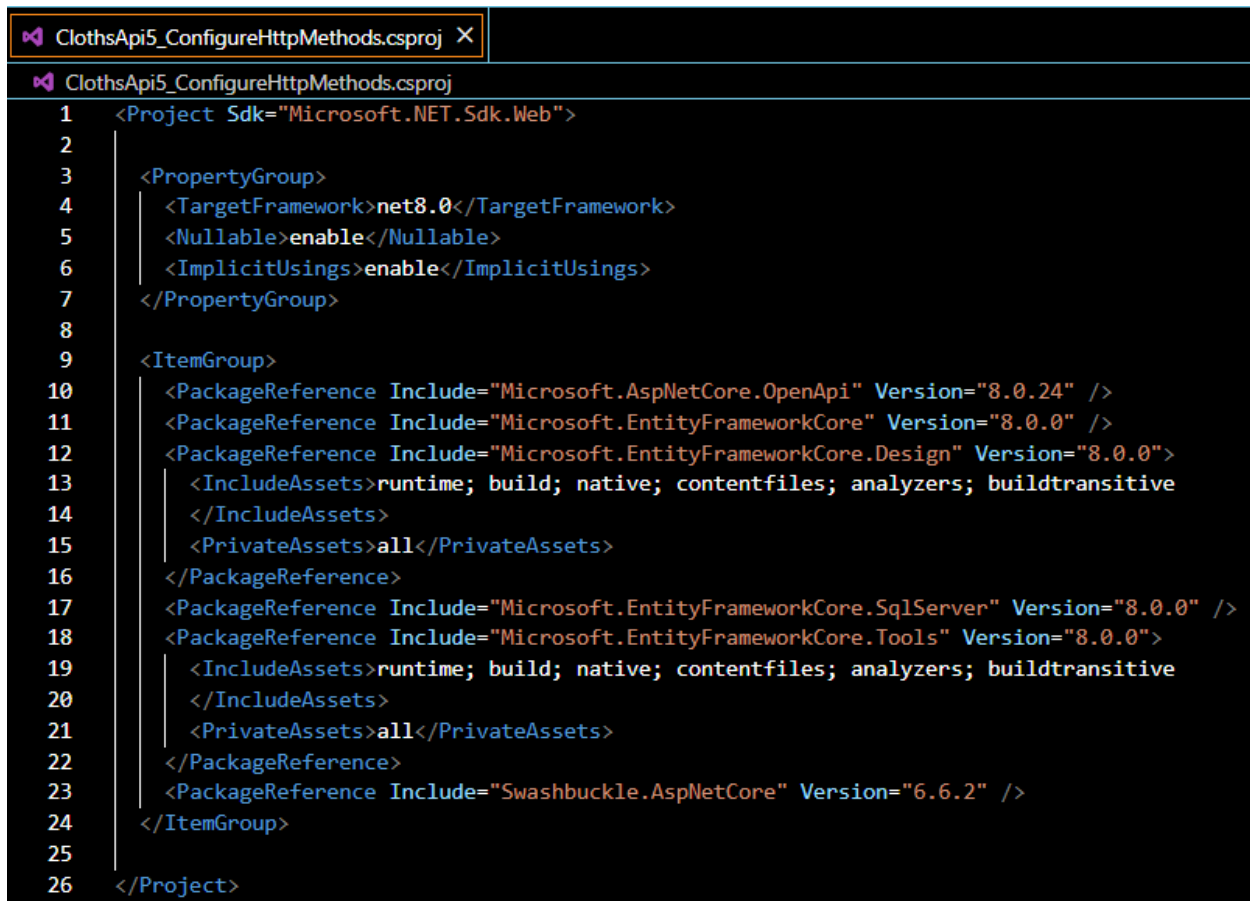
Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,



```
1 <Project Sdk="Microsoft.NET.Sdk.Web">
2
3   <PropertyGroup>
4     <TargetFramework>net8.0</TargetFramework>
5     <Nullable>enable</Nullable>
6     <ImplicitUsings>enable</ImplicitUsings>
7   </PropertyGroup>
8
9   <ItemGroup>
10    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
11    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
12    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
13      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
14    </IncludeAssets>
15    <PrivateAssets>all</PrivateAssets>
16    </PackageReference>
17    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
18    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
19      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
20    </IncludeAssets>
21    <PrivateAssets>all</PrivateAssets>
22    </PackageReference>
23    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
24  </ItemGroup>
25
26 </Project>
```

Step 1: Create Cloth Model

Path: ClothsApi5_ConfigureHttpMethods/Models/Cloth.cs

```
C# Cloth.cs X
Models > Cloth.cs > ...
1 namespace ClothsApi5_ConfigureHttpMethods.Models
2 {
3     0 references
4     public class Cloth
5     {
6         0 references
7         public int Id { get; set; }
8         0 references
9         public string Name { get; set; } = string.Empty;
10        0 references
11        public string Category { get; set; } = string.Empty;
12        0 references
13        public string Size { get; set; } = string.Empty;
14        0 references
15        public string Color { get; set; } = string.Empty;
16        0 references
17        public decimal Price { get; set; }
18        0 references
19        public bool InStock { get; set; }
20    }
21 }
```

Step 2: Create Cloth Service

Path: ClothsApi5_ConfigureHttpMethods/Services/ClothService.cs

```
C# ClothService.cs X
Services > ClothService.cs > ...
1 using ClothsApi5_ConfigureHttpMethods.Models;
2
3 namespace ClothsApi5_ConfigureHttpMethods.Services
4 {
5     0 references
6     public static class ClothService
7     {
8         2 references
9         private static readonly List<Cloth> _cloths = new()
10         {
11             new Cloth {
12                 Id = 1,
13                 Name = "Cotton T-Shirt",
14                 Category = "Men", Size = "M",
15                 Color = "Blue", Price = 599,
16                 InStock = true
17             },
18             new Cloth {
19                 Id = 2,
20                 Name = "Denim Jacket",
21                 Category = "Women",
22                 Size = "L",
23                 Color = "Black",
24                 Price = 2499,
25                 InStock = true
26             },
27             new Cloth {
28                 Id = 3,
29                 Name = "Formal Shirt",
30                 Category = "Men",
31                 Size = "XL",
32                 Color = "White",
33                 Price = 1299,
34                 InStock = false
35             },
36             new Cloth {
37                 Id = 4,
38                 Name = "Sports Shoes",
39                 Category = "Men",
40                 Size = "9",
41                 Color = "Grey",
42                 Price = 3499,
43                 InStock = true
44             },
45             new Cloth {
46                 Id = 5,
47                 Name = "Silk Saree",
48                 Category = "Women",
49                 Size = "Free",
50                 Color = "Red",
51                 Price = 4999,
52                 InStock = true
53             }
54         };
55     }
56 }
```

```

52         new Cloth {
53             Id = 6,
54             Name = "Kids Hoodie",
55             Category = "Kids",
56             Size = "S",
57             Color = "Yellow",
58             Price = 899,
59             InStock = true
60         },
61         new Cloth {
62             Id = 7,
63             Name = "Leather Belt",
64             Category = "Accessories",
65             Size = "32",
66             Color = "Brown",
67             Price = 399,
68             InStock = false
69         }
70     };
71
72     0 references
73     public static async Task<List<Cloth>> GetAllClothsAsync()
74     {
75         await Task.Delay(50);
76         return _cloths;
77     }
78
79     0 references
80     public static async Task<Cloth?> GetClothByIdAsync(int id)
81     {
82         await Task.Delay(30);
83         return _cloths.FirstOrDefault(c => c.Id == id);
84     }
85 }

```

Step 3: Create Middleware (Approach 1)

Path:

ClothsApi5_ConfigureHttpMethods/Middleware/HttpMethodMiddleware.cs

```
HttpMethodMiddleware.cs X
Middleware > C# HttpMethodMiddleware.cs > ...
1  using System.Text.Json;
2
3  namespace ClothsApi5_ConfigureHttpMethods.Middleware
4  {
5      // MIDDLEWARE - Runs before routing (earliest in pipeline)
6      // 1 reference
7      public class HttpMethodMiddleware
8      {
9          // 2 references
10         private readonly RequestDelegate _next;
11         // 3 references
12         private readonly string[] _allowedMethods;
13
14         // 0 references
15         public HttpMethodMiddleware(RequestDelegate next, string[] allowedMethods)
16         {
17             _next = next;
18             _allowedMethods = allowedMethods;
19         }
20
21         // 0 references
22         public async Task InvokeAsync(HttpContext context)
23         {
24             // Check if current HTTP method is allowed
25             if (!_allowedMethods.Contains(context.Request.Method))
26             {
27                 // Return 405 immediately - request never reaches controller
28                 context.Response.StatusCode = StatusCodes.Status405MethodNotAllowed;
29                 context.Response.ContentType = "application/json";
30
31                 var response = new
32                 {
33                     Code = 405,
34                     Message = $"Http Method '{context.Request.Method}' is not allowed." +
35                             $" Allowed methods: {string.Join(", ", _allowedMethods)}",
36                     Source = "HttpMethodMiddleware"
37                 };
38
39                 var responseJson = JsonSerializer.Serialize(response);
40                 await context.Response.WriteAsync(responseJson);
41                 return;
42             }
43
44             await _next(context); // Continue to next middleware
45         }
46     }
47 }
```

Step 4: Create ActionFilter (Approach 2)

Path: ClothsApi5_ConfigureHttpMethods/Filters/HttpMethodFilter.cs

```
HttpMethodFilter.cs X
Filters > C# HttpMethodFilter.cs > ...
1  using Microsoft.AspNetCore.Mvc;
2  using Microsoft.AspNetCore.Mvc.Filters;
3
4  namespace ClothsApi5_ConfigureHttpMethods.Filters
5  {
6      // ACTION FILTER APPROACH - Runs after routing, before action executes
7      public class HttpMethodFilter : IActionFilter
8      {
9          // 3 references
10         private readonly string[] _allowedMethods;
11
12         // 0 references
13         public HttpMethodFilter(string[] allowedMethods)
14         {
15             _allowedMethods = allowedMethods;
16         }
17     }
18 }
```

```
16  public void OnActionExecuting(ActionExecutingContext context)
17  {
18      // Check if current HTTP method is allowed
19      if (!_allowedMethods.Contains(context.HttpContext.Request.Method))
20      {
21          var response = new
22          {
23              Code = 405,
24              Message = $"HTTP Method '{context.HttpContext.Request.Method}' is "+
25              $"not allowed. Allowed methods: {string.Join(", ", _allowedMethods)}",
26              Source = "HttpMethodFilter",
27              Controller = context.Controller?.GetType().Name,
28              Action = context.ActionDescriptor.DisplayName
29          };
30
31          context.Result = new ObjectResult(response)
32          {
33              StatusCode = 405
34          };
35      }
36  }
37
38  // 0 references
39  public void OnActionExecuted(ActionExecutedContext context)
40  {
41      // Not used in this example
42  }
43 }
```

Step 5: Program.cs (Register Both)

Path: ClothsApi5_ConfigureHttpMethods/Program.cs

```
Program.cs X
Program.cs
1  using ClothsApi5_ConfigureHttpMethods.Middleware;
2  using ClothsApi5_ConfigureHttpMethods.Filters;
3
4  var builder = WebApplication.CreateBuilder(args);
5
6  builder.Services.AddControllers(options =>
7  {
8      // =====
9      // APPROACH 2: Register ActionFilter Globally
10     // This filter will run after routing, before every action
11     // =====
12     options.Filters.Add(new HttpMethodFilter(new[] { "GET", "POST", "DELETE" }));
13 });
14
15 builder.Services.AddEndpointsApiExplorer();
16 builder.Services.AddSwaggerGen();
17
18 var app = builder.Build();
19
20 if (app.Environment.IsDevelopment())
21 {
22     app.UseSwagger();
23     app.UseSwaggerUI();
24 }
25
26 app.UseHttpsRedirection();
27 app.UseAuthorization();
28
29 // =====
30 // APPROACH 1: Register Middleware
31 // This runs BEFORE routing (earliest in pipeline)
32 // =====
33 var allowedMethods = new[] { "GET", "POST", "DELETE" };
34 app.Use(async (context, next) =>
35 {
36     var middleware = new HttpMethodMiddleware(next, allowedMethods);
37     await middleware.InvokeAsync(context);
38 });
39
40 app.MapControllers();
41
42 app.Run();
43
```


Step 6: Controller

Path: ClothsApi5_ConfigureHttpMethods/Controller/ClothController.cs

```
ClothController.cs X
Controllers > ClothController.cs > ...
1  using Microsoft.AspNetCore.Mvc;
2  using ClothsApi5_ConfigureHttpMethods.Models;
3  using ClothsApi5_ConfigureHttpMethods.Services;
4
5  namespace ClothsApi5_ConfigureHttpMethods.Controllers
6  {
7      [ApiController]
8      [Route("api/[controller]")]
9      public class ClothController : ControllerBase
10     {
11         // GET /api/cloth - Allowed (GET)
12         [HttpGet]
13         public async Task<ActionResult<List<Cloth>>> GetAllCloths()
14         {
15             var cloths = await ClothService.GetAllClothsAsync();
16             return Ok(cloths);
17         }
18
19         // GET /api/cloth/3 - Allowed (GET)
20         [HttpGet("{id}")]
21         public async Task<ActionResult<Cloth>> GetClothById(int id)
22         {
23             var cloth = await ClothService.GetClothByIdAsync(id);
24             if (cloth == null)
25                 return NotFound($"Cloth with ID {id} not found");
26             return Ok(cloth);
27         }
28
29         // POST /api/cloth - Allowed (POST)
30         [HttpPost]
31         public async Task<ActionResult<Cloth>> CreateCloth([FromBody] Cloth newCloth)
32         {
33             // This is just a demo - not full implementation
34             return Ok(new { Message = "POST request received", Data = newCloth });
35         }
36     }
```

```

37 // PUT /api/cloth/5 - NOT Allowed (PUT not in allowed list)
38 [HttpPut("{id}")]
   0 references
39 public async Task<ActionResult<Cloth>> UpdateCloth()
40 {
41     return Ok(new {
42         Message = "PUT request received (should be blocked by Middleware/ActionFilter)"
43     });
44 }
45
46 // PATCH /api/cloth/5 - NOT Allowed (PATCH not in allowed list)
47 [HttpPatch("{id}")]
   0 references
48 public async Task<ActionResult<Cloth>> PatchCloth()
49 {
50     return Ok(new {
51         Message = "PATCH request received (should be blocked by Middleware/ActionFilter)"
52     });
53 }
54
55 // DELETE /api/cloth/5 - Allowed (DELETE)
56 [HttpDelete("{id}")]
   0 references
57 public async Task<IActionResult> DeleteCloth(int id)
58 {
59     return Ok(new { Message = $"DELETE request received for ID {id}" });
60 }
61 }
62 }

```

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli.

We have to navigate to the exact project folder

(ClothsApi5_ConfigureHttpMethods) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi5_ConfigureHttpMethods> dotnet build
Determining projects to restore...
All projects are up-to-date for restore.
ClothsApi5_ConfigureHttpMethods -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi5_ConfigureHttpMethods\bin\Debug\net8.0\ClothsApi5_ConfigureHttpMethods.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.96
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi5_ConfigureHttpMethods>
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi5_ConfigureHttpMethods> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/hot-reload.
💡 Press "Ctrl + R" to restart.
dotnet watch 🏗 Building...
Determining projects to restore...
All projects are up-to-date for restore.
ClothsApi5_ConfigureHttpMethods -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi5_ConfigureHttpMethods\bin\Debug\net8.0\ClothsApi5_ConfigureHttpMethods.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5282
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi5_ConfigureHttpMethod
s
[]
```

 **Swagger**
Supported by SMARTBEAR

Select a definition ClothsApi5_ConfigureHttpMethods v1

ClothsApi5_ConfigureHttpMethods

1.0 OAS 3.0

<http://localhost:5282/swagger/v1/swagger.json>

Cloth

GET

/api/Cloth

⌵

POST

/api/Cloth

⌵

GET

/api/Cloth/{id}

⌵

PUT

/api/Cloth/{id}

⌵

PATCH

/api/Cloth/{id}

⌵

DELETE

/api/Cloth/{id}

⌵

Schemas

Cloth >

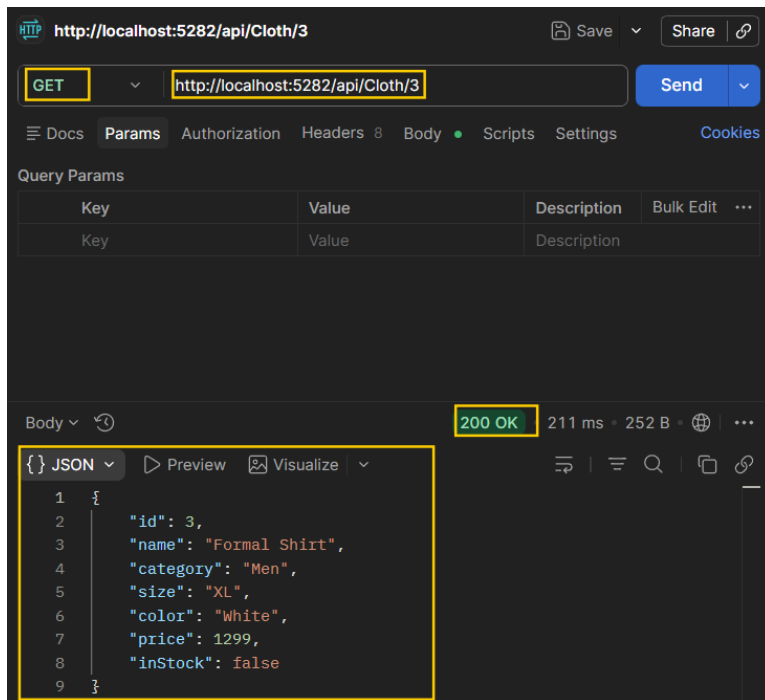
ALLOWED METHODS: GET, POST, DELETE

[N.B: We are going to use **Postman** here to test our APIs]

Test 1: GET Request (Allowed)

Request: GET http://localhost:5282/api/Cloth/3

Response (200 OK):



The screenshot shows an HTTP client interface with the following details:

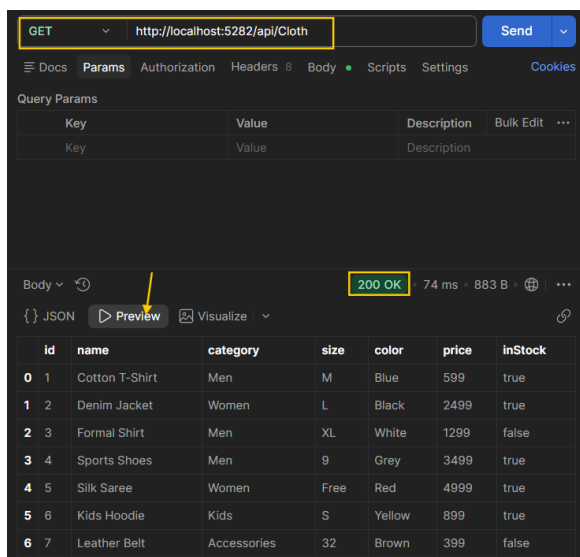
- Method: GET
- URL: http://localhost:5282/api/Cloth/3
- Status: 200 OK
- Time: 211 ms
- Size: 252 B
- Body: JSON

```
{
  "id": 3,
  "name": "Formal Shirt",
  "category": "Men",
  "size": "XL",
  "color": "White",
  "price": 1299,
  "inStock": false
}
```

Test 2: GET All (Allowed)

Request: GET http://localhost:5282/api/Cloth

Response (200 OK):



The screenshot shows an HTTP client interface with the following details:

- Method: GET
- URL: http://localhost:5282/api/Cloth
- Status: 200 OK
- Time: 74 ms
- Size: 883 B
- Body: JSON

The 'Preview' button is highlighted with a yellow arrow. The response is a JSON array of clothing items, displayed in a table below:

	id	name	category	size	color	price	inStock
0	1	Cotton T-Shirt	Men	M	Blue	599	true
1	2	Denim Jacket	Women	L	Black	2499	true
2	3	Formal Shirt	Men	XL	White	1299	false
3	4	Sports Shoes	Men	9	Grey	3499	true
4	5	Silk Saree	Women	Free	Red	4999	true
5	6	Kids Hoodie	Kids	S	Yellow	899	true
6	7	Leather Belt	Accessories	32	Brown	399	false

Test 3: POST Request (Allowed)

Request: POST http://localhost:5282/api/Cloth

Request Body:

Content-Type: application/json

```
{  
  "name": "Test Shirt",  
  "price": 599  
}
```

Response (200 OK):

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:5282/api/Cloth
- Body Tab:** Selected, showing the request body in JSON format:

```
1 {  
2   "name": "Test Shirt",  
3   "price": 599  
4 }
```
- Status:** 200 OK (highlighted in green), with response time of 60 ms and size of 282 B.
- Response Body:** The response is shown in JSON format:

```
1 {  
2   "message": "POST request received",  
3   "data": {  
4     "id": 0,  
5     "name": "Test Shirt",  
6     "category": "",  
7     "size": "",  
8     "color": "",  
9     "price": 599,  
10    "inStock": false  
11  }  
12 }
```

Test 4: DELETE Request (Allowed)

Request: DELETE http://localhost:5282/api/Cloth/5

Response (200 OK):

The screenshot shows a REST client interface with a dark theme. At the top, the HTTP method is set to 'DELETE' and the URL is 'http://localhost:5282/api/Cloth/5'. A 'Send' button is on the right. Below the URL bar, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', and 'Settings'. The 'Params' tab is active, showing a table for 'Query Params' with columns 'Key', 'Value', 'Description', and 'Bulk Edit'. The 'Body' tab is also visible, showing a '200 OK' status. The response body is displayed in JSON format, showing a message: 'DELETE request received for ID 5'.

DELETE http://localhost:5282/api/Cloth/5 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK 18 ms • 194 B • ...

{ } JSON Preview Visualize

```
1 {
2   "message": "DELETE request received for ID 5"
3 }
```

Test 5: PUT Request (NOT Allowed – Should be Blocked)

Request: PUT http://localhost:5282/api/Cloth/5

Request Body:

Content-Type: application/json

```
{  
  "name": "Updated Shirt",  
  "price": 699  
}
```

Response (405 Method Not Allowed):

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://localhost:5282/api/Cloth/5
- Body Tab:** Selected, showing the request body in JSON format:

```
{  
  "name": "Updated Shirt",  
  "price": 699  
}
```
- Response:** 405 Method Not Allowed (39 ms, 284 B). The response body is shown in JSON format:

```
{  
  "Code": 405,  
  "Message": "Http Method 'PUT' is not allowed. Allowed methods: GET,  
  "Source": "HttpMethodMiddleware"
```


Test 6: PATCH Request (NOT Allowed – Should be Blocked)

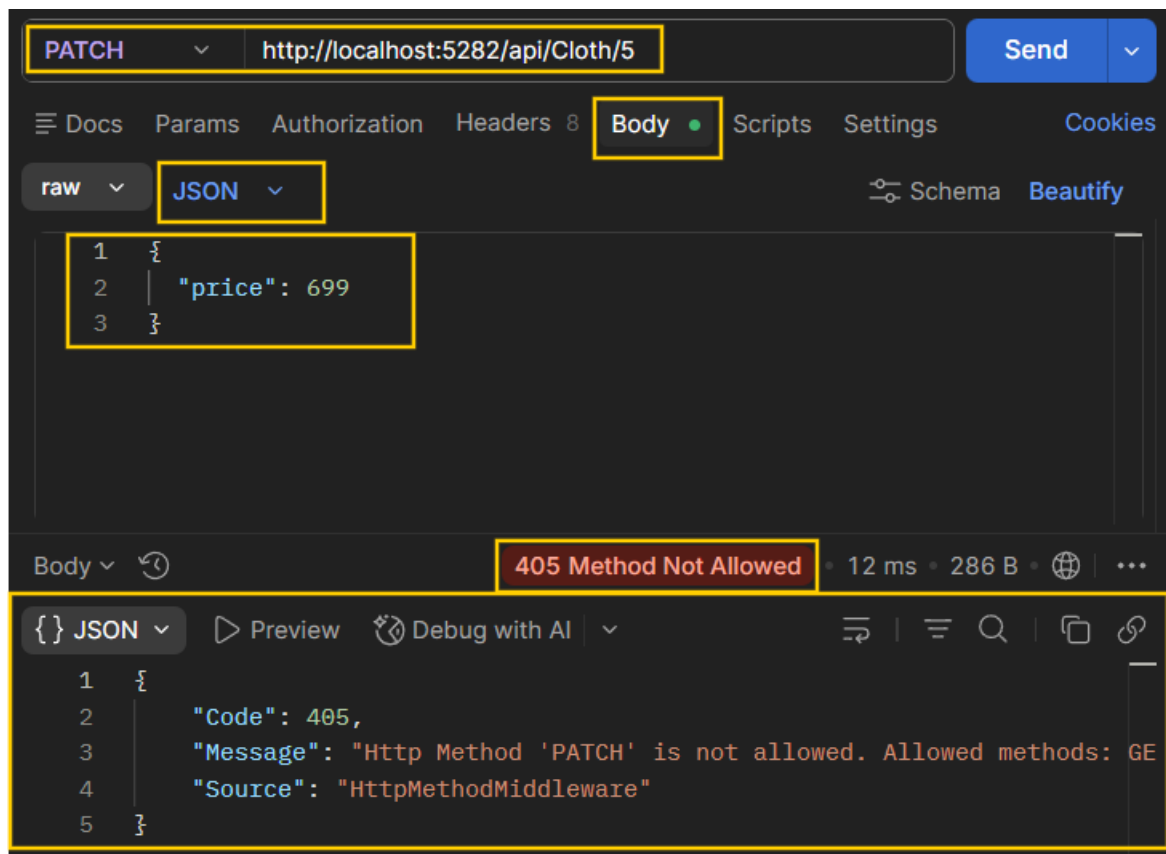
Request: PATCH http://localhost:5282/api/Cloth/5

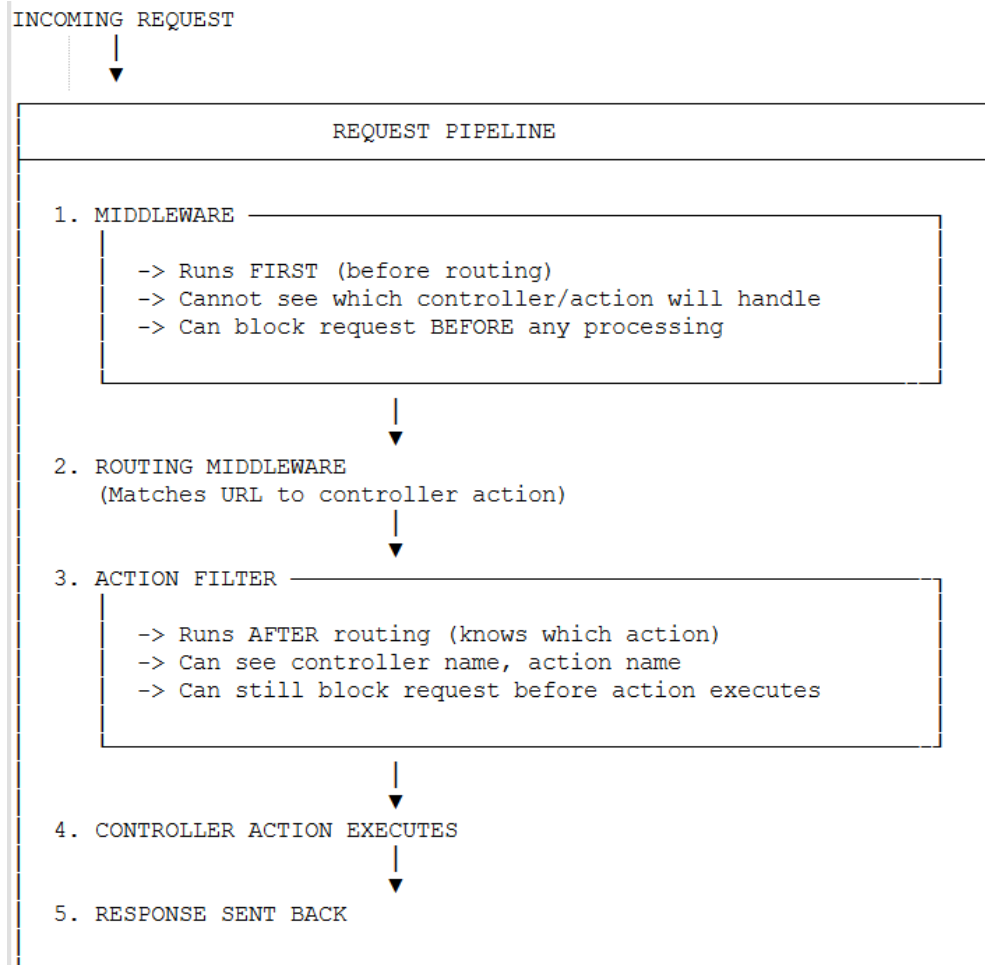
Request Body:

Content-Type: application/json

```
{  
  "price": 699  
}
```

Response (405 Method Not Allowed):





Comparison: Middleware vs ActionFilter

Execution Order	Middleware	ActionFilter
Execution Order	Before routing	After routing, before action
When it runs	Very early in pipeline	After routing is complete
Access to route data	No	Yes (can see controller/action names)
Performance	Slightly faster	Slightly slower
Source identifier	"HttpMethodMiddleware"	"HttpMethodFilter"
When to use	Simple global blocking	When we need action context

Key Takeaways

Concept	Explanation
Middleware	Runs earliest, best performance, good for simple global rules
ActionFilter	Runs after routing, can access controller/action context
Global Configuration	One place to define allowed methods for ENTIRE API
405 Status Code	Returned when HTTP method is not supported
Order Matters	Middleware runs first (PUT/PATCH never reach ActionFilter)